

Universidad EAFIT

Estructuras de Datos y Algoritmos — 2026/01

Práctica Final Integradora

Análisis de la red vial roadNet-PA con algoritmos sobre grafos

Equipo:

Nash Díaz

Daniel Osorio

Alejandro Restrepo

Mayo de 2026

1. Introducción

El análisis de redes de transporte es uno de los problemas clásicos de grafos con mayor impacto práctico. Aplicaciones como la planificación de rutas en sistemas de navegación, la optimización logística y el estudio de la conectividad urbana operan directamente sobre grafos que representan millones de intersecciones y tramos viales. Los algoritmos clásicos del curso (BFS, DFS, Dijkstra, MST) son las herramientas básicas con las que estos sistemas resuelven sus consultas en tiempo razonable.

En este proyecto trabajamos sobre el dataset roadNet-PA (Pennsylvania Road Network), parte de la colección SNAP del Stanford Network Analysis Project. Este grafo es uno de los benchmarks de referencia del 9th DIMACS Implementation Challenge on Shortest Paths (2006), lo que permite verificar los resultados obtenidos contra valores publicados. El grafo tiene 1.088.092 nodos y 1.541.898 aristas no dirigidas, con un diámetro reportado de 782 saltos. Estas propiedades lo hacen ideal para evaluar el comportamiento de los algoritmos sobre datos reales y para forzar decisiones de diseño que en grafos pequeños no aparecen, como la inviabilidad de la matriz de adyacencia.

El programa implementa tres módulos seleccionables por argumento de línea de comandos: análisis estructural (Módulo A), camino mínimo punto a punto sobre 10 consultas predefinidas (Módulo B) y análisis de un subgrafo inducido con cálculo de MST y verificación DAG (Módulo C).

2. Descripción del dataset

El archivo roadNet-PA.txt contiene el grafo en formato SNAP, con líneas de comentario prefijadas con # y líneas de datos del tipo FromNodeId<TAB>ToNodeId. Los identificadores de nodo no son consecutivos. El archivo lista cada arista no dirigida dos veces, una en cada dirección, lo que se debe tener en cuenta al cargar para evitar contar el doble.

La siguiente tabla compara las estadísticas obtenidas por nuestra implementación con los valores publicados por SNAP:

Propiedad	Valor SNAP	Valor obtenido
Nodos	1.088.092	1.088.092
Aristas no dirigidas	1.541.898	1.541.898
Componente conexa principal	1.087.562	1.087.562
Componentes conexas totales	—	206
Grado promedio	2,83	2,8341
Nodo de mayor grado	—	674.502 (grado 9)

Propiedad	Valor SNAP	Valor obtenido
Diámetro	782	772 (estimado)

La coincidencia exacta en nodos, aristas y componente conexa principal verifica que el cargador interpreta el archivo correctamente y que el manejo de aristas duplicadas funciona. El diámetro se estimó con la heurística de doble BFS, descrita en la sección 3, que produce una cota inferior del diámetro real. La diferencia respecto a 782 es del 1,3%, consistente con que el doble BFS no garantiza el valor exacto pero sí una aproximación muy cercana cuando el grafo es esencialmente conexo.

Sobre las propiedades estructurales, roadNet-PA es un grafo extremadamente disperso (densidad de 2,83 aristas por nodo), casi completamente conexo (la componente principal contiene el 99,95% de los nodos) y con un diámetro grande relativo al número de nodos. Estas tres propiedades son típicas de redes viales reales: las intersecciones tienen pocos vecinos directos, la red es prácticamente conexa porque está pensada para que cualquier punto sea alcanzable, y el diámetro crece aproximadamente con la raíz del número de nodos por la naturaleza geográfica plana de la red.

3. Diseño e implementación

3.1 Representación del grafo

Adoptamos la lista de adyacencia como representación principal, implementada como `std::vector<std::vector<Edge>>`, donde `Edge` es una estructura simple que contiene el nodo destino y el peso. La justificación es de memoria: una matriz de adyacencia para 1.088.092 nodos requeriría una tabla de $1.088.092 \times 1.088.092$ enteros de 32 bits, es decir, aproximadamente 4,7 TB de RAM ($1.088.092^2 \times 4 \text{ bytes} \approx 4,73 \times 10^{12} \text{ bytes}$). Esto excede en órdenes de magnitud cualquier computador estándar y descarta la matriz de plano. La lista de adyacencia, en cambio, ocupa aproximadamente 50–80 MB para este grafo y cabe sin problemas en cualquier máquina moderna.

3.2 Reindexación de nodos

Como los IDs originales no son consecutivos, mantenemos dos estructuras de mapeo: un `unordered_map<int,int>` que traduce IDs originales a índices internos en $O(1)$ promedio, y un `vector<int>` que traduce el índice interno al ID original también en $O(1)$. Las consultas externas (como las 10 del Módulo B) se especifican con IDs originales del dataset; el programa los traduce al entrar y los re-traduce al salir. Esta separación hace que las estructuras internas (lista de adyacencia, vectores de distancias y predecesores) puedan usar índices contiguos $0..N-1$, lo que es eficiente en memoria y en acceso.

3.3 Asignación de pesos y manejo de duplicados

El dataset original no incluye pesos. Para simular distancias viales asignamos un peso entero aleatorio uniforme entre 1 y 10 a cada arista no dirigida en el momento de la carga, usando `srand(42)` antes de iniciar la lectura para garantizar reproducibilidad. Como el archivo lista cada arista no dirigida dos veces, mantenemos un `set<pair<int,int>>` con los pares ordenados ya vistos y descartamos los duplicados. Así cada arista se inserta una sola vez, con un único peso aleatorio que se aplica a ambas direcciones.

3.4 Pseudocódigo de Dijkstra con heap mínimo

```
funcion Dijkstra(G, source, target):
    dist[v] = infinito para todo v
    parent[v] = -1 para todo v
    settled[v] = falso para todo v
    dist[source] = 0
    pq = priority_queue minima de pares (distancia, nodo)
    pq.push((0, source))

    mientras pq no este vacia:
        (d, u) = pq.pop()
        si settled[u] entonces continuar
        settled[u] = verdadero
        si u == target entonces retornar dist, parent

        para cada arista (u, v, w) en G.neighbors(u):
            nd = d + w
            si nd < dist[v]:
                dist[v] = nd
                parent[v] = u
                pq.push((nd, v))

    retornar dist, parent
```

La marca `settled` evita procesar entradas obsoletas que quedan en la cola tras una relajación posterior. El corte temprano `if u == target` permite que Dijkstra termine apenas el destino se extrae del heap, sin recorrer toda la componente.

3.5 Pseudocódigo de estimación de diámetro por doble BFS

```
funcion EstimarDiametro(G, start):
    farthest_a = BFS_NodoMasLejano(G, start)
    diameter = BFS_DistanciaMaxima(G, farthest_a)
    retornar diameter
```

La heurística aprovecha que en grafos casi conexos, hacer BFS desde un nodo arbitrario tiende a ubicar un extremo del diámetro como el nodo más lejano. Hacer un segundo BFS desde ese nodo entrega una cota inferior muy ajustada del diámetro real.

4. Análisis de complejidad

BFS sobre lista de adyacencia. Cada nodo se inserta y se extrae de la cola exactamente una vez ($O(V)$) y cada arista se examina exactamente una vez ($O(E)$), de modo que el costo total es $O(V + E)$. Para roadNet-PA esto es del orden de 4 millones de operaciones, perfectamente manejable.

Dijkstra con heap mínimo. Cada nodo puede insertarse en el heap $O(V)$ veces en el peor caso, pero solo se procesa al extraerlo si no está marcado como settled. Cada operación del heap (insertar o extraer mínimo) cuesta $O(\log V)$. En total, el algoritmo realiza $O((V + E) \log V)$ operaciones. Para roadNet-PA esto da aproximadamente 4 millones $\times \log(10^6) \approx 80$ millones de operaciones elementales, lo que en la práctica corresponde a tiempos del orden de cientos de milisegundos cuando hay que recorrer toda la componente, y muy por debajo cuando el destino está cerca y el corte temprano permite terminar antes.

La ventaja decisiva de la lista sobre la matriz para este grafo es que cada iteración de BFS o Dijkstra solo examina los vecinos reales del nodo actual, lo que aprovecha la dispersión del grafo. Con matriz, examinar los vecinos costaría $O(V)$ por nodo, llevando ambos algoritmos a $O(V^2) \approx 10^{12}$ operaciones, además del problema irresoluble de los 4,7 TB de memoria.

5. Resultados de consultas P2P

La siguiente tabla muestra los resultados de las 10 consultas predefinidas del Módulo B, con pesos generados con `srand(42)`:

Q	Origen	Destino	Dist.	Saltos	Nodos Dij.	Nodos BFS	t_Dij. (ms)	t_BFS (ms)
Q01	1	500.000	529	101	67.461	54.713	17,82	10,25
Q02	100	1.000.000	2.332	484	1.015.460	1.008.999	222,99	45,68
Q03	50.000	750.000	2.557	545	932.705	925.233	280,20	54,36
Q04	200.000	800.000	611	124	93.044	88.414	23,40	9,03
Q05	300.000	100.000	3.098	666	1.028.438	1.051.816	230,13	51,78
Q06	1	1.087.562	404	95	36.422	47.273	10,50	4,35
Q07	500.000	1	529	101	102.246	90.936	36,42	9,64
Q08	250.000	600.000	1.492	300	785.003	758.054	206,16	39,20
Q09	10.000	900.000	889	169	182.950	156.513	44,46	9,60
Q10	400.000	150.000	805	167	354.064	352.068	110,49	23,45

La columna Dist. mide la distancia ponderada (suma de pesos del camino mínimo) y la columna Saltos mide el número de aristas del camino con menos saltos. Como los pesos están entre 1 y 10, la distancia ponderada es típicamente de 5 a 6 veces el número de saltos, y los datos lo confirman (529 vs 101, 2.332 vs 484, etc.). Esta relación es consistente con un peso promedio cercano a 5,5, exactamente el valor esperado para una distribución uniforme entre 1 y 10.

En términos de nodos explorados, BFS y Dijkstra terminan recorriendo cantidades similares cuando el destino está lejos (Q02, Q03, Q05, Q08), porque ambos tienden a expandirse en frente hacia él. En las consultas donde el destino está más cerca (Q01, Q06), Dijkstra explora algo menos por el corte temprano combinado con el orden de extracción por distancia.

Los tiempos de Dijkstra son en promedio 4 a 5 veces mayores que los de BFS, lo cual es consistente con el factor $\log V$ adicional que introduce el heap. Esta es la diferencia teórica esperada: Dijkstra paga ese sobrecosto a cambio de respetar los pesos. En consultas donde el destino está lejos, ambos llegan a ejecutar entre 200 y 280 ms para Dijkstra y entre 40 y 55 ms para BFS, tiempos coherentes con el tamaño del grafo.

Los caminos completos para Q01 (529 unidades de distancia, 101 saltos) y Q06 (404 unidades, 95 saltos) se reconstruyeron usando el arreglo parent siguiendo los predecesores desde el destino hasta el origen, y se exportaron a results/camino_Q01.txt y results/camino_Q06.txt.

6. Análisis del subgrafo

El Módulo C construye el subgrafo inducido por la unión de los nodos de los caminos de Q01 y Q06. El subgrafo resultante tiene 225 nodos y 228 aristas. Esto es un grafo pequeño y casi-árbol: un árbol sobre 225 nodos tendría exactamente 224 aristas, así que aquí solo hay 4 aristas adicionales que cierran ciclos.

MST. Aplicamos Kruskal con Union-Find por rango y compresión de caminos. El MST resultante tiene 224 aristas (uno menos que el número de nodos, como debe ser un árbol cobertor) y peso total 927. Eso da un peso promedio por arista en el MST de 4,14, ligeramente por debajo del promedio teórico de pesos (5,5), lo cual es esperable porque Kruskal selecciona las aristas más livianas primero.

DAG. Para verificar si el subgrafo es DAG hicimos DFS con clasificación de aristas (tree edges, back edges, forward edges, cross edges). Las back edges son las que van de un descendiente a un ancestro en el árbol DFS y son las que indican ciclos en grafos dirigidos. En nuestra ejecución obtuvimos 189 tree edges y 0 back edges, lo que significa que el subgrafo es DAG cuando se interpreta con la orientación que tiene en la lista de adyacencia.

Es importante señalar el contexto: una arista no dirigida (u, v) se almacena en la lista de adyacencia como dos aristas dirigidas $(u \rightarrow v)$ y $(v \rightarrow u)$. Si el subgrafo se considerara con todas estas orientaciones simultáneamente, cualquier subgrafo con al menos una arista produciría una back edge trivial. Sin embargo, el resultado de 0 back edges en la pasada de DFS muestra que las orientaciones almacenadas, al recorrerlas en el orden de la adyacencia, no forman ciclos.

dirigidos no triviales. Geométricamente, esto es consistente con que el subgrafo está formado por dos caminos mínimos que comparten algunos nodos en sus extremos pero no forman bucles cerrados con orientación consistente.

El subgrafo se exportó a results/subgrafo_caminos.txt en formato de lista de adyacencia compatible con el de la Evaluación 03.

7. Conclusiones

Trabajar con un grafo real de un millón de nodos cambia la perspectiva sobre las decisiones de diseño que en grafos pequeños son irrelevantes. La diferencia entre lista y matriz de adyacencia, que en un grafo de 100 nodos es solo cosmética, en roadNet-PA es la diferencia entre un programa que carga en dos segundos y un programa que es físicamente imposible de ejecutar. La elección de heap binario sobre arreglo lineal para Dijkstra, que en un grafo pequeño no se nota, pasa de $O(V^2)$ a $O((V+E) \log V)$, lo cual sí se nota en milisegundos vs minutos.

También aprendimos que las heurísticas pragmáticas tienen mucho valor cuando el cómputo exacto es prohibitivo. Calcular el diámetro exacto de roadNet-PA requeriría hacer BFS desde cada uno de los aproximadamente 1 millón de nodos, mientras que el doble BFS entrega una estimación con apenas dos pasadas y se queda a un 1,3% del valor real. Una primera versión del proyecto, que estimaba el diámetro con un único BFS desde el nodo de mayor grado, daba un resultado muy alejado (alrededor de 590), porque el nodo de mayor grado del grafo no es necesariamente extremo del diámetro. Cambiar a doble BFS fue una corrección importante que ilustra cómo la elección del nodo de partida es decisiva.

Sobre limitaciones, el peso aleatorio uniforme entre 1 y 10 es una simplificación grande respecto a las distancias viales reales, que tienen distribuciones muy distintas. Sin embargo, la simplificación permite reproducibilidad exacta entre equipos y mantiene el foco del trabajo en los algoritmos. Una extensión natural sería usar pesos derivados de coordenadas geográficas, lo que también habilita técnicas más avanzadas como A* con heurísticas geométricas. Otra limitación es que el subgrafo del Módulo C es muy pequeño (225 nodos), por lo que el MST y la verificación DAG no permiten conclusiones generales sobre la estructura del grafo completo. Su valor está en mostrar que las técnicas funcionan y que se integran limpiamente con los caminos del Módulo B.

8. Referencias

Leskovec, J. y Krevl, A. (2014). SNAP Datasets: Stanford Large Network Dataset Collection. Recuperado de <http://snap.stanford.edu/data>

Demetrescu, C., Goldberg, A. V. y Johnson, D. S. (Eds.). (2009). The Shortest Path Problem: Ninth DIMACS Implementation Challenge. American Mathematical Society. <http://www.dis.uniroma1.it/challenge9/>

Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2009). Introduction to Algorithms (3.^a ed.). MIT Press. Capítulos sobre BFS, DFS, Dijkstra y MST.

Material de la asignatura Estructuras de Datos y Algoritmos, Universidad EAFIT, 2026/01.

9. Herramientas utilizadas

Durante el desarrollo del proyecto, el equipo recurrió a Claude (Anthropic) como apoyo en partes específicas del trabajo. El uso fue selectivo y orientado a destrabar puntos en los que la documentación del curso o nuestro propio conocimiento eran insuficientes. En particular:

- **Estimación del diámetro.** Inicialmente intentamos estimar el diámetro con un único BFS desde el nodo de mayor grado, lo que producía una cota muy alejada del valor real (alrededor de 590 frente al valor publicado de 782). Consultamos sobre alternativas y se nos sugirió la heurística de doble BFS, que implementamos y verificamos contra el valor SNAP, pasando de un error del 24% a un error del 1,3%.
- **Manejo de duplicados en el dataset.** Una versión temprana del cargador no contemplaba que el archivo SNAP lista cada arista no dirigida dos veces, lo que duplicaba el conteo de aristas. Con apoyo de la IA identificamos el problema y diseñamos la solución usando un `set<pair<int,int>>` ordenado.
- **Clasificación de aristas en DFS.** El concepto de tree edges y back edges para la verificación DAG estaba en el material del curso, pero su aplicación correcta a un grafo no dirigido cargado como dirigido tenía sutilezas que aclaramos consultando a la IA. La interpretación final del resultado y la decisión de documentar honestamente cómo se obtiene están escritas con nuestras propias palabras.
- **Redacción del informe.** Discutimos la estructura general del informe y revisamos algunas secciones para mejorar claridad y concisión, pero la organización, los argumentos y las interpretaciones son del equipo.

En todos los casos, el código que se incluye en el repositorio fue revisado, modificado y probado por los integrantes del equipo, y puede ser explicado por cualquiera de nosotros. Las decisiones de diseño relevantes (lista de adyacencia, reindexación, manejo de pesos con `srand(42)`, corte temprano en Dijkstra, doble BFS para diámetro) se discutieron en grupo antes de implementarse.

NOTA: Tenemos dos repositorios, esto porque el proyecto lo empezamos temprano, pero a la hora verificar minuciosamente vimos que habían unos errores en temas de compilación y resultados, y el encargado del repositorio en ese momento no tenía conexión, así que creamos un segundo repositorio con todo solucionado.

Primer repo: <https://github.com/Martesito/EDA-Proyecto-final-.git>

Segundo repo:

https://github.com/NashVale/EDA_PF_NashDiaz_DanielOsorio_AlejandroRestrepo.git